NVIDIA JETSON AGX THOR DEVELOPER KIT

The Ultimate Platform for Humanoid Robotics

Setup and Demo Guide

CONTENTS

SETUP AND GETTING STARTED
HARDWARE-IN-THE-LOOP (HIL)- GR00T N1 ON JETSON THOR
VIDEO SEARCH AND SUMMARIZATION
RUN THE GENERATIVE AI BENCHMARKS
APPENDIX
RESOURCES

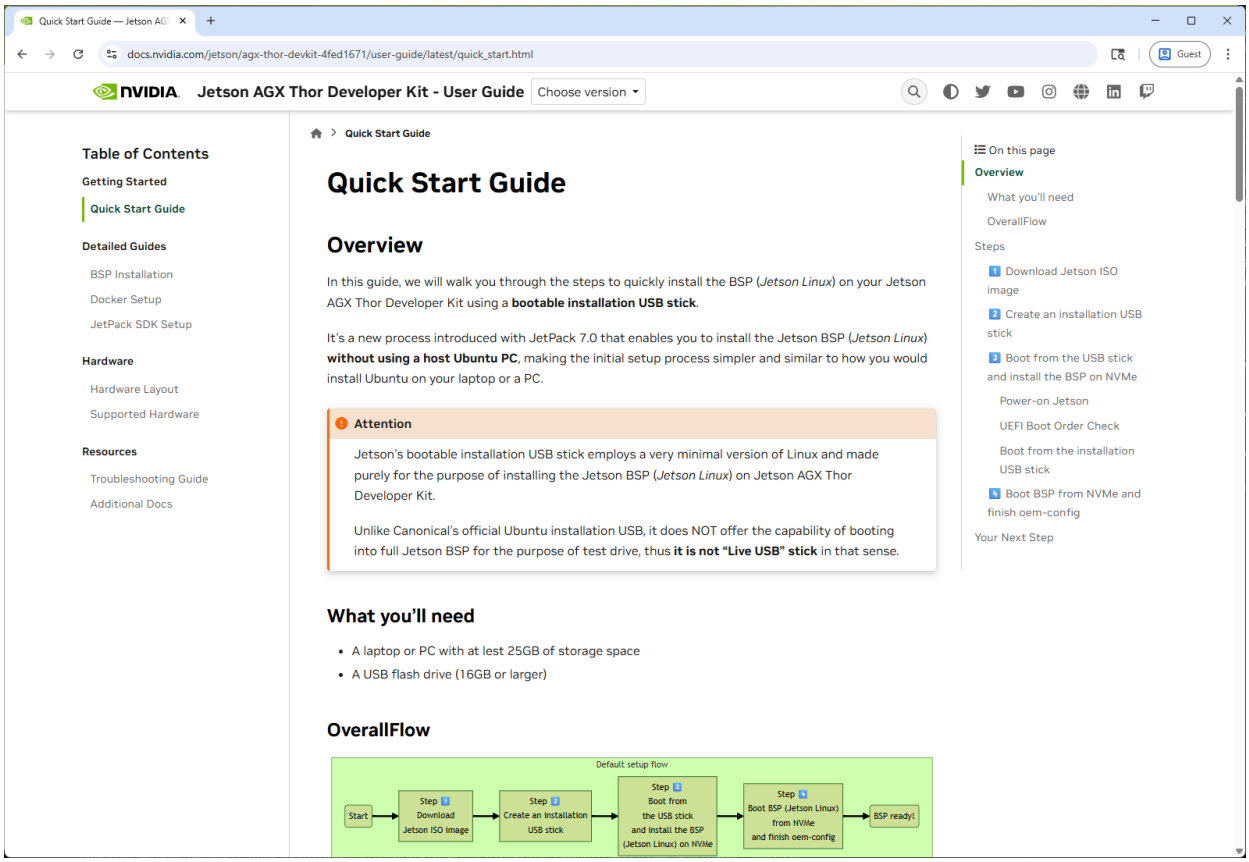
Congratulations on receiving your new NVIDIA AGX Thor Developer Kit! Your Developer Kit is the gateway to building next-generation AI and robotics applications. This guide will help you set it up and dive straight into demos that showcase its unmatched performance and intelligence.

Your NVIDIA Jetson Thor Developer Kit runs on JetPack 7 and supports the latest AI compute stack. Let's get started by preparing your developer kit.

Note: All software featured in this guide is offered as Early Access and may be subject to change.

SETUP AND GETTING STARTED

Please follow the instructions on the online user guide to create a USB drive using the JetPack 7 ISO image. We will then use this USB drive to install Jetson Linux 38.1 (BSP) on your developer kit.



Link: https://docs.nvidia.com/jetson/agx-thor-devkit-4fed1671/user-guide/latest/quick_start.html

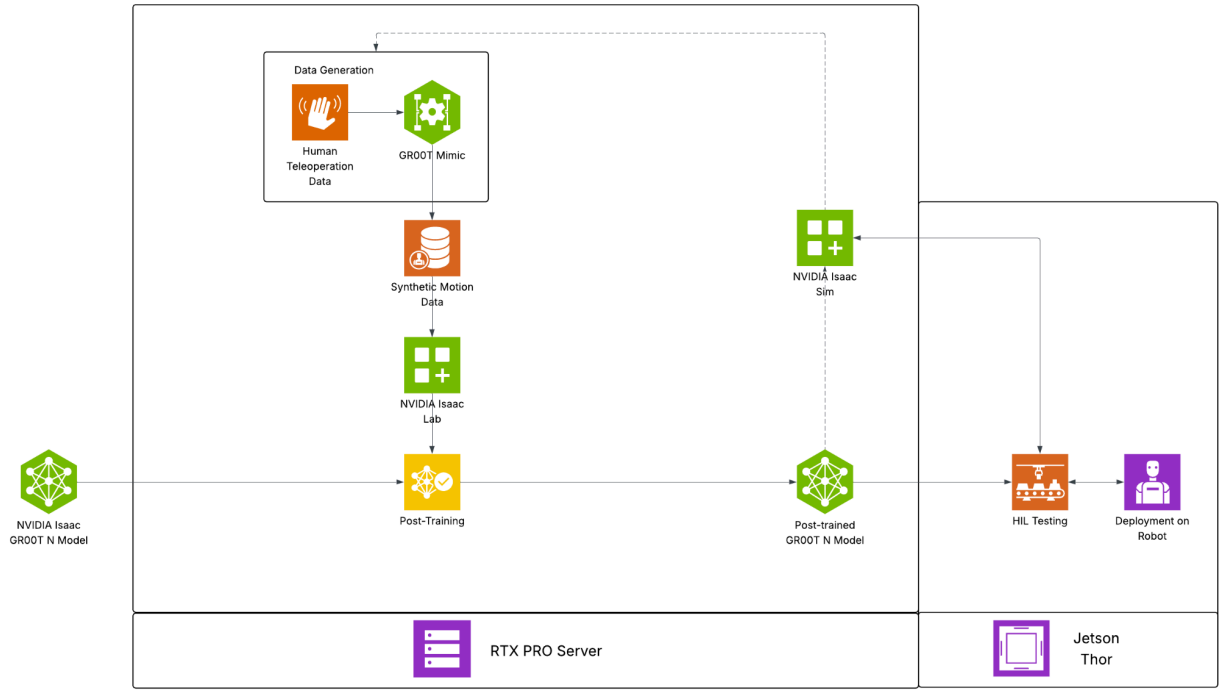
HARDWARE-IN-THE-LOOP (HIL)- GR00T N1 ON JETSON THOR

In this demo, we showcase a Hardware-in-the-Loop (HIL) setup where the GR00T N1 foundation model runs directly on Jetson AGX Thor to control a robot in Isaac Sim. The robot performs a coordinated “nut-pouring” task- picking up a beaker of metallic nuts, dispensing a single nut into a bowl, and placing the bowl on a scale. This setup allows us to evaluate real on-device behavior, including timing, latency, and resource utilization, on the same class of hardware designed for deployment.

To make this HIL run possible—and reliable on Jetson AGX Thor—we use NVIDIA’s Isaac GR00T simulation-first workflow. We capture a small set of teleoperation demos in Isaac Sim/Lab, expand them with Omniverse-based Isaac GR00T blueprints into large volumes of physically plausible trajectories, then post-train the GR00T N1 Vision-Language-Action (VLA) model for the target embodiment and task. In practice, NVIDIA generated >750K synthetic trajectories in ~11 hours and saw ~40% higher performance when mixing synthetic with real data—making data collection practical while improving robustness. To help teams start fast, NVIDIA also provides open SimReady datasets (Physical AI collection) and the GR00T N1-2B weights on Hugging Face, so this simulation-first, post-train-and-deploy workflow can be reproduced and adapted to your robot and task.

The workflow summarized

Before setting up and running the HIL example, let’s walk through the end-to-end workflow that brought us here.



STEP 1: Selecting a suitable pretrained model

For the humanoid robot nut-pouring task, we chose the GR00T N1 Vision-Language-Action (VLA) foundation model, pretrained for versatile perception and control.

STEP 2: Generating data for post-training

In Isaac Sim and Isaac Lab, we captured precise demonstrations of the full sequence- grasping, pouring, and placing, by teleoperating the simulated robot while recording motions, images, and outcomes. Using the NVIDIA Isaac GR00T Blueprint for synthetic manipulation motion generation, we scaled the initial 10 recorded demonstrations to 1,000 through synthetic variations such as different starting poses and object placements, improving the model's robustness across diverse scenes. All demonstrations were stored in a standard HDF5 format and then converted into the GR00T-Lerobot dataset structure for use in our training pipeline.

STEP 3: Post-training with generated data

Using the synthesized dataset, we fine-tuned the GR00T N1 model for the nut-pouring task, adapting it to the sequence and constraints of the operation—ensuring stable grasps, controlled pouring angle and rate, and precise placement.

STEP 4: Validate in Software, Then on Hardware

Before moving to physical tests, we validated the model entirely within Isaac Sim, enabling rapid iteration, logic verification, and risk reduction, a process known as Software-in-the-Loop (SIL). Once the model passed SIL validation, we transitioned to Hardware-in-the-Loop (HIL), the configuration used in this demo.

Hardware-in-the-Loop

Software-in-the-Loop (SIL) runs both the model and the simulated environment entirely on a workstation. This approach is fast, safe, and well-suited for debugging algorithms and validating task logic. However, it cannot fully reveal how the system will behave on actual deployment hardware.

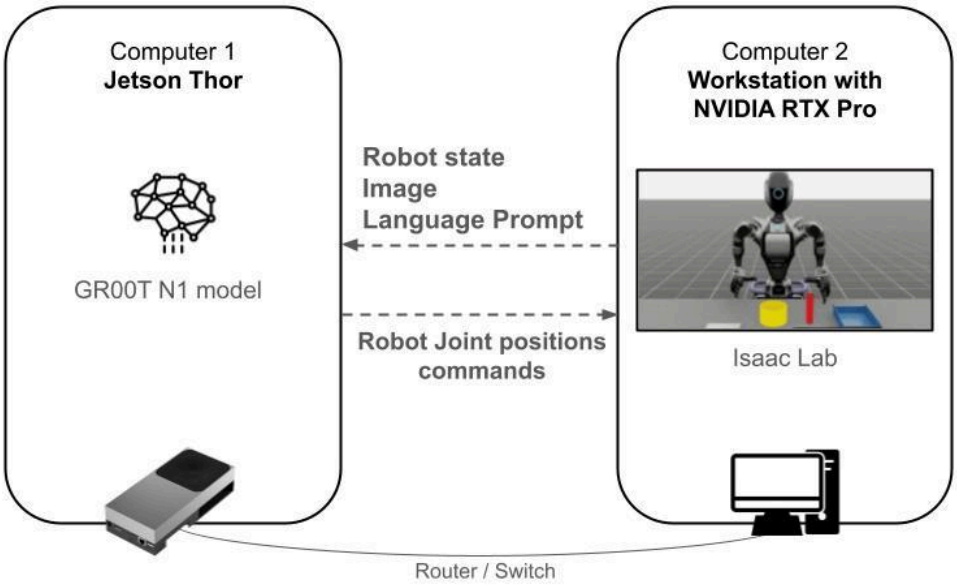
Hardware-in-the-Loop (HIL) retains the simulated environment but runs the robot's "brain" on the real device—here, the Jetson AGX Thor. The simulator streams observations to the Jetson, which executes the model, then returns actions to the simulator. This setup exposes true on-device performance, including policy inference latency, end-to-end loop timing, and resource utilization, as well as integration nuances that SIL cannot capture.

Together, SIL and HIL answer complementary questions: SIL verifies whether the model or policy makes the correct decisions, while HIL assesses if those decisions can be executed at the necessary speed on real hardware, operating within realistic constraints.

Hardware-in-the-Loop (HIL) offers significant benefits by revealing hardware constraints early, such as compute and thermal limits, network behavior, middleware nuances, and resource contention between GPU and CPU tasks, while maintaining a safe and repeatable environment. This early visibility enables precise measurement, shortens integration time, and provides

critical insights that guide both model and system design, transforming promising simulation results into reliable, deployable capabilities.

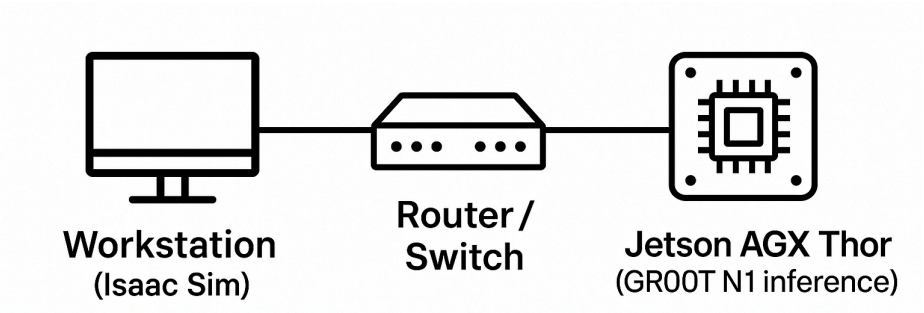
The diagram below illustrates the setup used in this demo.



In this demo, we'll run the GR00T N1 policy on Jetson AGX Thor while Isaac Sim runs on the workstation for sensing and visualization. You'll start the Thor inference container, then the workstation container; Isaac Sim will stream observations to Thor, and Thor will return joint commands to execute the nut-pouring sequence.

System setup explained
This demo uses the following hardware, software, and network configurations.

- Hardware
  - Workstation PC running Isaac Sim
  - Jetson AGX Thor Developer Kit for on-device inference
- Software
  - Workstation materials `sim-robot-environment`
  - Jetson container `thor-groot-inference`
- Network
  - ROS 2 between workstation and Jetson on the same local network- direct USB-C or shared Ethernet.



Next, We'll guide you through starting inference on Thor, launching Isaac Sim, and confirming the ROS 2 connection.

Jetson AGX Thor Setup

Prerequisites

1. Configure Docker to enable GPU access for containers by using the NVIDIA Container Runtime. Then, restart the Docker service to apply the changes:

```
Shell
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

Running the inference container on Jetson AGX Thor

Next, launch the thor-groot-inference container on the Thor system. This container runs inference with the fine-tuned GR00T N1 model specifically designed for the nut-pouring task.

1. Download the container

```
Shell
wget
https://international.download.nvidia.com/JetsonThorReview/thor-groot-inference
.tar.gz
```

After the download has been completed please run the following command:

```
Shell
sha256sum thor-groot-inference.tar.gz
```

The hash value you obtain should match the hash shown in the image below; if it doesn't, please run wget again—your download may not have completed successfully.

```
d24ecc2bde1d7645a55bef91f2a07929801cf4af99871a7ccf6c8c6ea66dac2e  thor-groot-inference.tar.gz
```

2. Load the container from the directory where it was downloaded, typically the ‘Downloads’ folder

```
Shell
cd Downloads
gunzip -k thor-groot-inference.tar.gz
docker load -i thor-groot-inference.tar
```

3. Run the container using the following flags

```
Shell
docker run --gpus all --runtime=nvidia --privileged -it --rm -u 0:0
--cap-add=SYS_PTRACE --cap-add SYS_ADMIN --cap-add DAC_READ_SEARCH
--security-opt seccomp=unconfined --name=$NAME --shm-size=256g --ulimit
memlock=-1 --ulimit stack=67108864 --ipc=host --network=host
thor-groot-inference
```

4. Wait a few moments, once your screen displays this, you’ve successfully entered the running container

```
root@jetson:/workspace/gr00t_inference_ros2#
```

Workstation Setup

Running the Isaac lab simulation container

Let’s launch the sim-robot-environment container on your workstation. This container runs the simulation environment featuring the humanoid and objects within Isaac Lab.

This demo has been validated on an Ubuntu 24.04 system equipped with an NVIDIA RTX A6000 GPU. It is also compatible with other systems outlined in the Isaac Sim System Requirements.

1. Download the container

```
Python
wget
https://international.download.nvidia.com/JetsonThorReview/workstation_material
s.zip
```

After the download has been completed please run the following command:

```
Shell
sha256sum workstation_materials.zip
```

The hash value you obtain should match the hash shown in the image below; if it doesn't, please run wget again—your download may not have completed successfully.

```
c2bd4536071ac17e407dfdfe48fbd7c2f78fc7a5c3d319f2f1c94d99b3d225c3  workstation_materials.zip
```

2. Load the container from the directory where it was downloaded, typically the 'Downloads' folder

```
Shell
cd Downloads
unzip workstation_materials.zip
gunzip -k sim-robot-environment.tar.gz

docker load -i sim-robot-environment.tar
```

3. Run the container using the following command

```
Shell
python run_container.py
```

4. Wait a few moments, once your screen displays this, you've successfully entered the running container

Verify Workstation–Thor Communication

By now, both containers should be running- one on Thor and the other on your workstation. Before starting the demo, let's verify that they can communicate with each other.

To verify this, run a ROS 2 topic publisher on your workstation and subscribe to the same topic on Thor to confirm cross-device communication.

Inside the workstation container, run the following command

```
Shell
ros2 run demo_nodes_py talker
```

You should see output messages being published to the topic, similar to following

```
File  Edit  View  Search  Terminal  Help
root@3132415-lcelt:/IsaacLab-Internal# ros2 run demo_nodes_cpp talker
[INFO] [1754329665.923692147] [talker]: Publishing: 'Hello World: 1'
[INFO] [1754329666.923651226] [talker]: Publishing: 'Hello World: 2'
[INFO] [1754329667.923651182] [talker]: Publishing: 'Hello World: 3'
[INFO] [1754329668.923717925] [talker]: Publishing: 'Hello World: 4'
[INFO] [1754329669.923651846] [talker]: Publishing: 'Hello World: 5'
[INFO] [1754329670.923649575] [talker]: Publishing: 'Hello World: 6'
[INFO] [1754329671.923647227] [talker]: Publishing: 'Hello World: 7'
[INFO] [1754329672.923826356] [talker]: Publishing: 'Hello World: 8'
[INFO] [1754329673.923644442] [talker]: Publishing: 'Hello World: 9'
[INFO] [1754329674.923665094] [talker]: Publishing: 'Hello World: 10'
[INFO] [1754329675.923643798] [talker]: Publishing: 'Hello World: 11'
[INFO] [1754329676.923640611] [talker]: Publishing: 'Hello World: 12'
[INFO] [1754329677.923638154] [talker]: Publishing: 'Hello World: 13'
[INFO] [1754329678.923640018] [talker]: Publishing: 'Hello World: 14'
[INFO] [1754329679.923630773] [talker]: Publishing: 'Hello World: 15'
[INFO] [1754329680.923636720] [talker]: Publishing: 'Hello World: 16'
[INFO] [1754329681.923627053] [talker]: Publishing: 'Hello World: 17'
[INFO] [1754329682.923633837] [talker]: Publishing: 'Hello World: 18'
[INFO] [1754329683.923714271] [talker]: Publishing: 'Hello World: 19'
[INFO] [1754329684.923671569] [talker]: Publishing: 'Hello World: 20'
[INFO] [1754329685.923631509] [talker]: Publishing: 'Hello World: 21'
[INFO] [1754329686.923628394] [talker]: Publishing: 'Hello World: 22'
```

Inside the Thor container, run the following command

```shell
ros2 topic echo /chatter
```

This will begin displaying the ROS 2 messages received from the talker node running on the workstation

```
File  Edit  View  Search  Terminal  Help
root@jetson-0722:/workspace/gr00t_inference_ros2# ros2 topic echo /chatter
data: 'Hello World: 7'
---
data: 'Hello World: 8'
---
data: 'Hello World: 9'
---
data: 'Hello World: 10'
---
data: 'Hello World: 11'
---
data: 'Hello World: 12'
---
data: 'Hello World: 13'
---
data: 'Hello World: 14'
---
data: 'Hello World: 15'
---
data: 'Hello World: 16'
---
data: 'Hello World: 17'
---
```

This test verifies that the Thor and workstation can communicate via ROS2. If you didn't see messages on the Thor:

- Verify that Thor is properly connected to your workstation
- Confirm that both devices are on the same local network
- Check that no firewall is blocking ROS 2 traffic- this issue can occur on secure or corporate or university networks

Now, let's proceed with running the HIL demo!

Running the HIL Demo

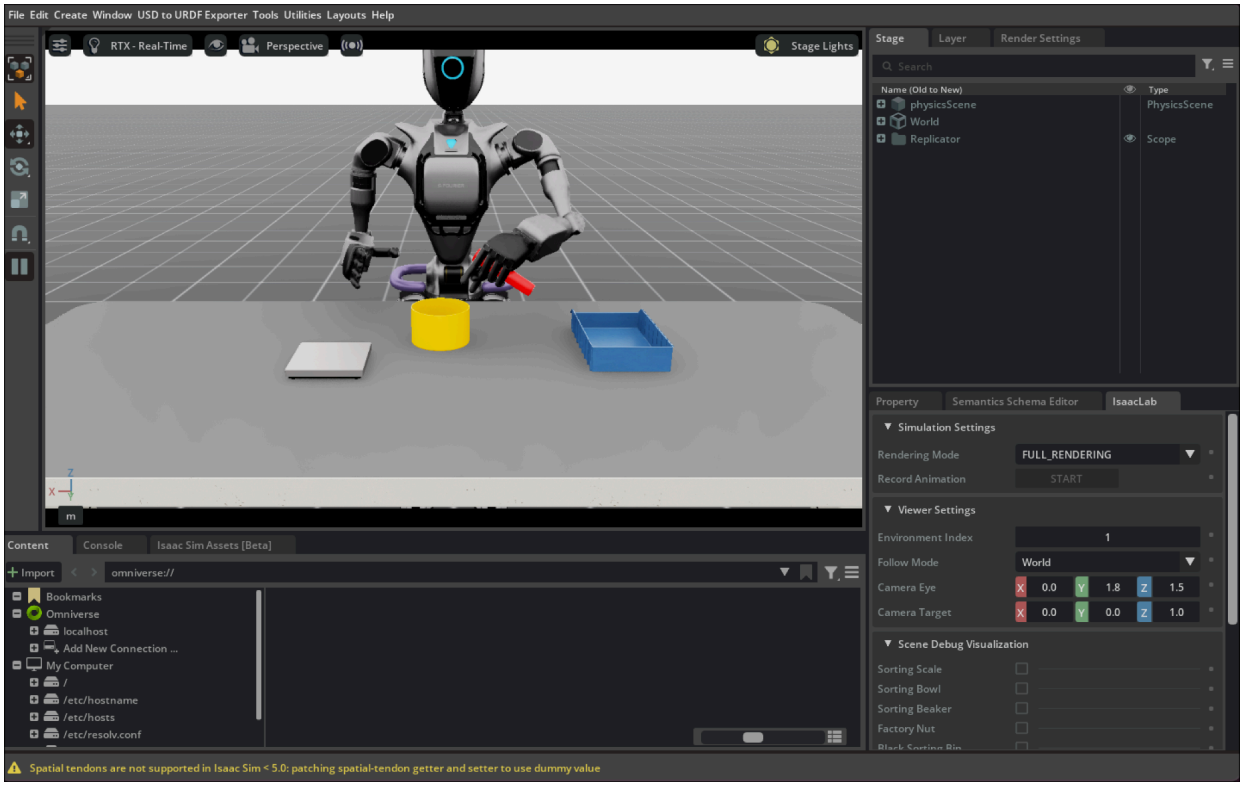
In the Thor container, run the following command to launch the GR00T inference node

```
Shell
run_inference
```

In the workstation container, run the following command to launch the Isaac Sim GUI, load the simulation, and start the task. **Note** that the first launch can take a few minutes while Isaac Sim loads.

```
Shell
./isaaclab.sh -p /isaaclab_node/isaaclab_ros_node.py
```

When it's ready, you'll see a window similar to the screenshot below.



You can watch the robot repeatedly perform the Nut Pouring task. After each completion, the simulation resets and the positions of the beaker, nuts, and bowl are randomly shifted within a 2 cm range before the next run. The checkpoint used here has been fine-tuned specifically for this range and achieves an accuracy of 90%. In other words, the robot successfully completes the task within this 2 cm variation 90% of the time. The task is deemed successful when the following conditions are met:

1. The factory nut is in the sorting bowl
2. The sorting beaker is placed in the sorting bin
3. The sorting bowl is placed on the sorting scale

Please note that the model is designed to handle scene variations within a 2 cm range, this is the expected generalization capability of the current checkpoint. Extending beyond this range (such as 10 cm shifts or swapping objects) requires additional post-training. Isaac Lab 2.2 provides a comprehensive tutorial on this process. While post-training is supported, it is outside the scope of this guide.

Visualizing input to GR00T

The GR00T N1 model takes three inputs: an image, the current instruction, and a task description prompt. The image is captured in real time by a camera mounted on the robot's neck.

Let's visualize the camera feed to gain a clearer understanding of how the model operates. From this perspective, the changes in object positions become more apparent each time the simulation resets.

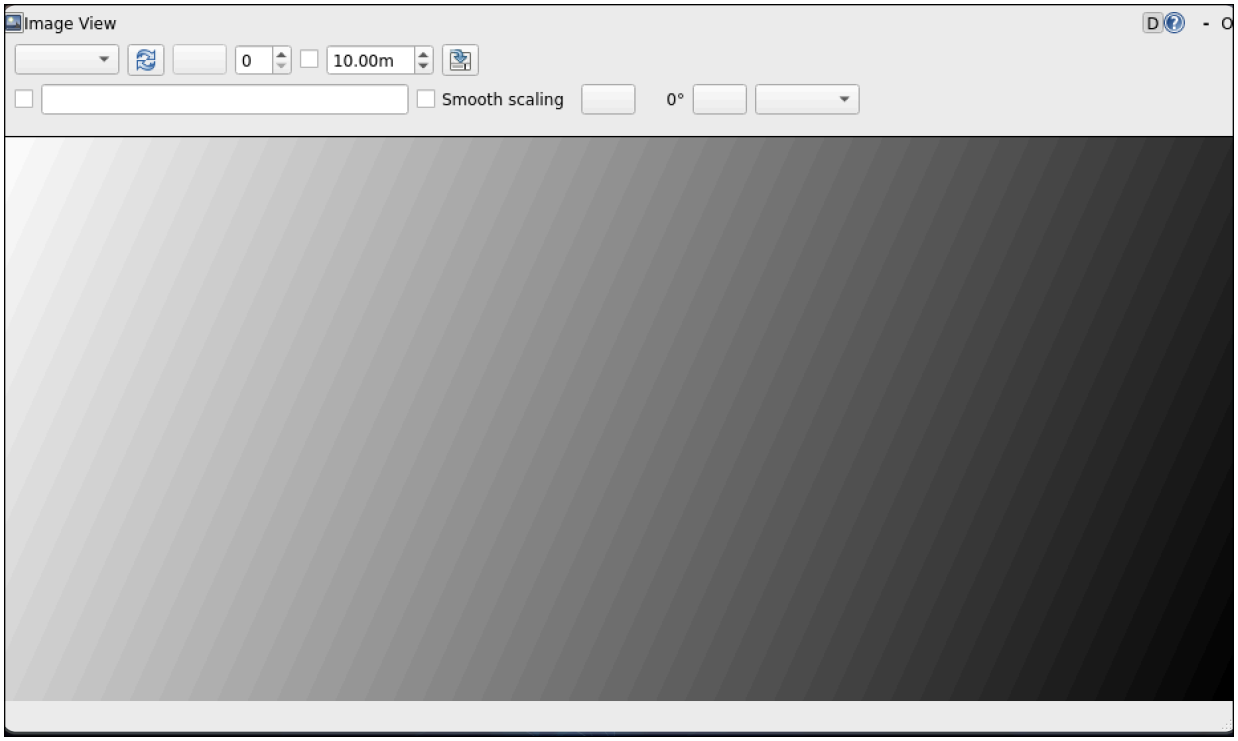
1. Open a new terminal on your workstation and run the script below to attach to the existing container. This will not start a new container but connect you to the one already running.

```
Shell
python run_container.py
```

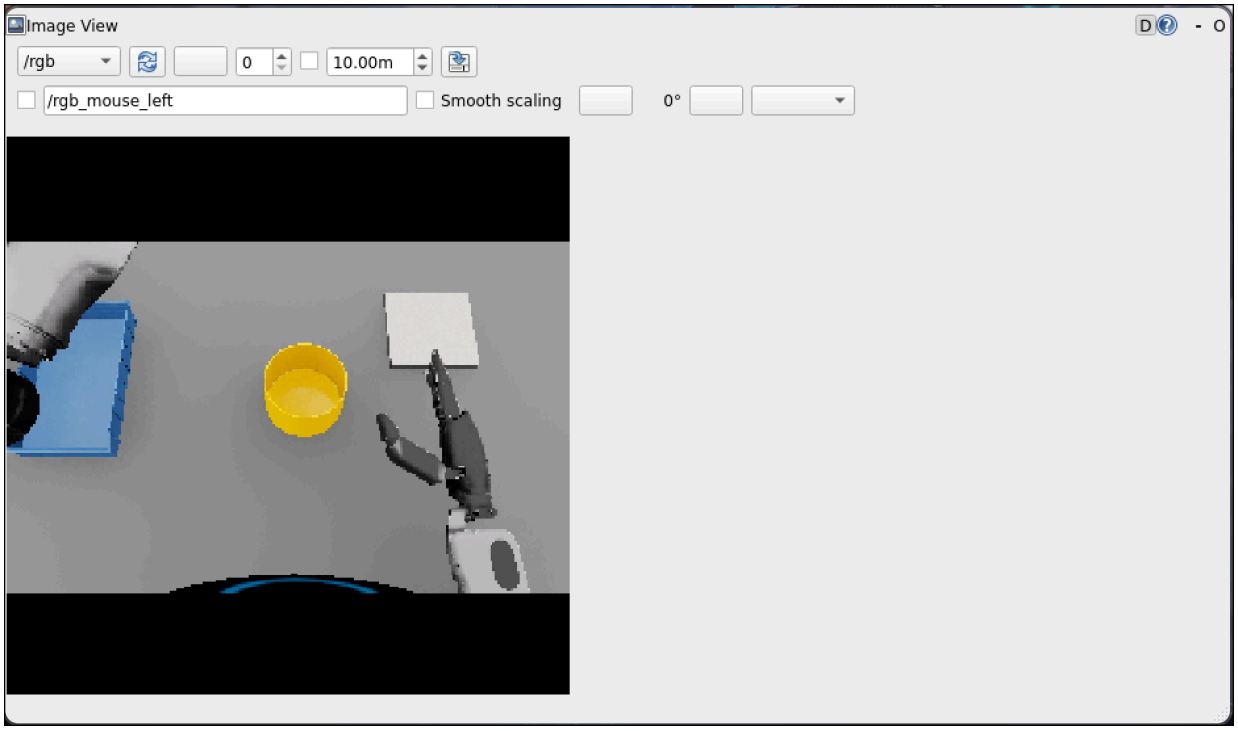
2. Once we are inside the container on this new terminal, run the following command to visualize the robot's camera stream.

```
None
ros2 run rqt_image_view rqt_image_view
```

3. This will open a GUI that appears as shown below.



4. Click on the top left box and choose **/rgb**. This will bring up the livestream from the robot's camera (image below). If the **/rgb** option does not show up, please click the refresh button right next to it.



Inspect ROS 2 communication

Let's examine the active ROS 2 topics and their published data to verify communication between Thor and the workstation, and to better visualize our setup.

1. Open a new terminal on your workstation and run the command below to attach to the existing container. This will not start a new container but connect you to the one already running.

```
Shell
python run_dev.py
```

2. View all ROS 2 topics.

```
Shell
ros2 topic list
```

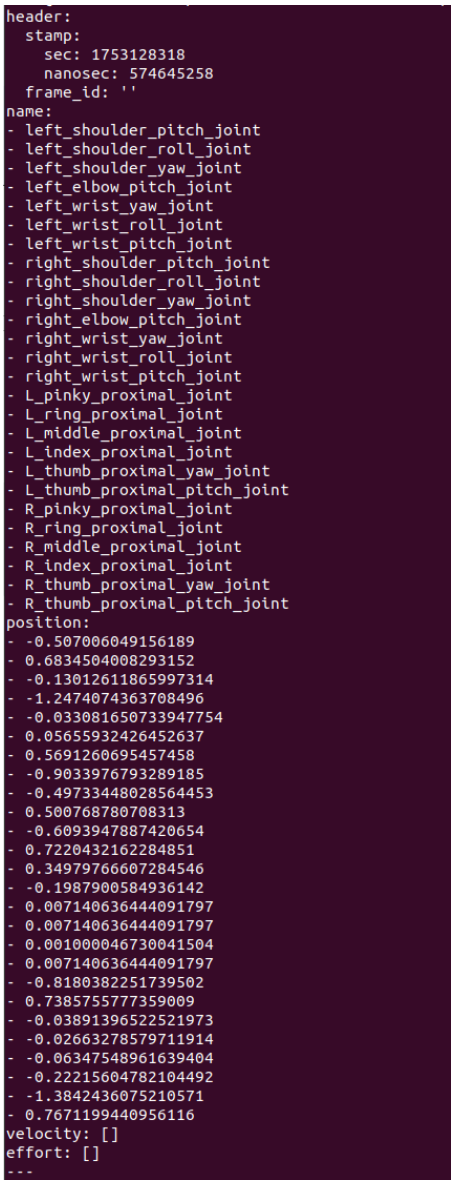
3. You should now see a screen similar to this.

```
/diagnostics
/joint_command
/joint_states
/parameter_events
/rgb
/rosout
```

4. In this example, the GR00T inference node (running on the Thor) uses **/rgb** and **/joint_states** as inputs. Camera frames are communicated on the **/rgb** topic, and the robot's current joint positions are communicated on the **/joint_states** topic. The GR00T model outputs actions on the **/joint_command** topic - these are the desired joint commands which are applied in the simulation on the next step.

5. You can print out information that a topic is publishing using the command (output image below).

```shell
ros2 topic echo /joint_command
```



In this way, the two processes communicate via ROS across two different systems:

1. Thor - running GR00T inference
2. Workstation - running environment simulation

This is the whole idea of HIL testing to easily validate hardware for your applications!

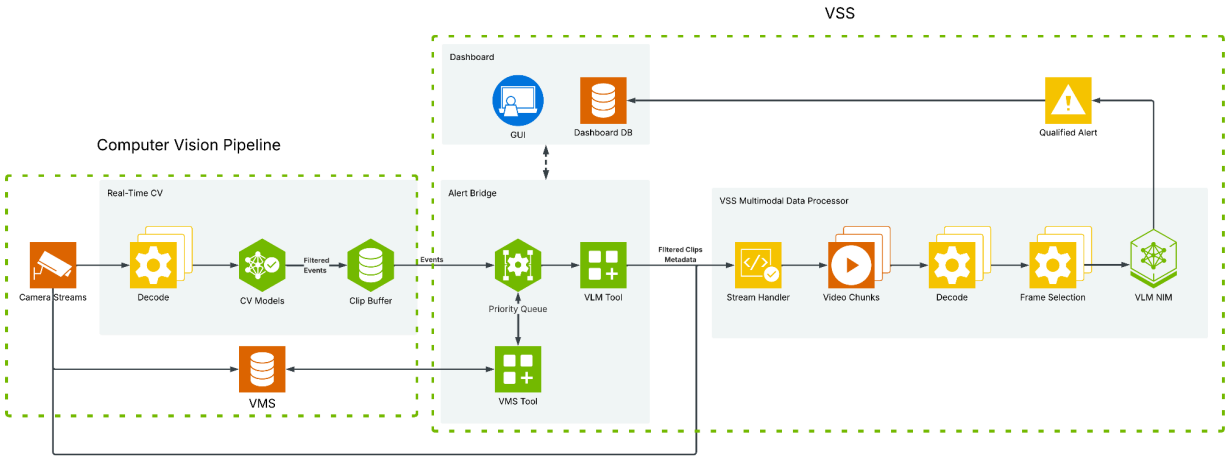
In this demo, we explored the GR00T model designed for general humanoid tasks, observed it running inference on the Thor, and reviewed the simulation environment setup in Isaac Sim. With Isaac Sim, you can configure various robots and environments for different tasks, enabling you to run GR00T and many other models seamlessly on your new Jetson Thor!

VIDEO SEARCH AND SUMMARIZATION

The Video Search and Summarization (VSS) demo showcases a new *event verification* feature, enabling VSS to act as an intelligent add-on to any existing computer vision (CV) pipeline.

While standard CV pipelines can detect objects like people appearing in view or apply analytics to identify events such as possible vehicle collisions, they often generate false positives and lack deeper scene understanding.

VSS enhances these pipelines by analyzing short video clips flagged by the CV system, verifying the detected events, and uncovering additional insights using that traditional methods may miss. VSS accomplishes this by using a Vision Language Model (VLM) - Cosmos Reason 1.



The following steps will walk you through setup and running a demo preview of the new VSS event verification feature. The demo consists of two main parts:

1) Computer Vision Pipeline
2) VSS

The computer vision pipeline is an example DeepStream detection pipeline using GroundDino that will take in a video, run detection and then output clips when the number of detected objects is greater than a set threshold. The purpose of this pipeline is to find the most important events from the video that need to be inspected by VSS with a VLM.

VSS will then process each small clip using the VLM Cosmos Reason 1 by answering a set of yes/no questions defined by the user. These responses are converted to True/False states for each question and can be used to generate low latency alerts to a user. In this demo they will be displayed on a web dashboard. Once the short clip has been processed by VSS, you can ask more detailed follow questions.

When the demo is launched, there will be two web UIs to interact with it. One Web UI will control the detection pipeline to generate clips of interest from the input video and the other will be the VSS UI to receive the VLM based alerts and insights on the short clips.

Setup

Software Prerequisites

If you have not done so already, install nvidia-jetpack.

```
Shell
echo "deb https://repo.download.nvidia.com/jetson/jetson-4fed1671 r38.1 main" |
sudo tee /etc/apt/sources.list.d/nvidia-l4t-apt-source.list > /dev/null
sudo apt update
sudo apt install nvidia-jetpack
```

Then configure docker to run without sudo.

```
Shell
sudo usermod -aG docker $USER
newgrp docker
```

Download demo content

Download Option 1 - Direct Download

Download all demo content directly to your system. This will include four containers, model weights and some configuration files. In total, the downloaded content will be 66GB.

```
Shell
cd ~/
mkdir vss_demo_downloads
```

Copy and paste the entire command block into your terminal to download all files.

```
Shell
wget --tries=0 --waitretry=5 -c -P ~/vss_demo_downloads/
https://international.download.nvidia.com/JetsonThorReview/alert-inspector-ui.t
ar.gz
```

```
wget --tries=0 --waitretry=5 -c -P ~/vss_demo_downloads/
https://international.download.nvidia.com/JetsonThorReview/Cosmos-Reason1.1-7B.
tar.gz
wget --tries=0 --waitretry=5 -c -P ~/vss_demo_downloads/
https://international.download.nvidia.com/JetsonThorReview/cv-ui.tar.gz
wget --tries=0 --waitretry=5 -c -P ~/vss_demo_downloads/
https://international.download.nvidia.com/JetsonThorReview/nv-cv-event-detector
.tar.gz
wget --tries=0 --waitretry=5 -c -P ~/vss_demo_downloads/
https://international.download.nvidia.com/JetsonThorReview/via-engine.tar.gz
wget --tries=0 --waitretry=5 -c -P ~/vss_demo_downloads/
https://international.download.nvidia.com/JetsonThorReview/vss_demo.tar.gz
```

If the download is interrupted or the connection is closed then rerun the entire download block to restart the download. The -c flag will ensure it picks up where it left off and skip any files that were fully downloaded. If there are frequent interruptions in the connection, you may need to run the download commands multiple times. Follow the steps below to verify if all demo assets downloaded successfully.

Download Option 2 - Google Drive

An alternative to the direct download commands is to manually download the files from google drive and then copy them into the ~/vss_demo_downloads folder on your Thor. You can find all the demo contents in this google drive folder.

In the vss_demo_downloads folder, verify you have the following content:

```
ubuntu@localhost:~/vss_demo_downloads$ ll
total 68585344
drwxrwxr-x  2 ubuntu ubuntu        4096 Aug  8 15:21 ./
drwxr-x--- 24 ubuntu ubuntu        4096 Aug  8 15:15 ../
-rw-------  1 ubuntu ubuntu 13114415963 Aug  8 15:16 alert-inspector-ui.tar.gz
-rw-rw-r--  1 ubuntu ubuntu 13306039377 Aug  8 15:16 Cosmos-Reason1.1-7B.tar.gz
-rw-------  1 ubuntu ubuntu 13404782699 Aug  8 15:18 cv-ui.tar.gz
-rw-------  1 ubuntu ubuntu 12172836111 Aug  8 15:19 nv-cv-event-detetor.tar.gz
-rw-------  1 ubuntu ubuntu 18233274916 Aug  8 15:21 via-engine.tar.gz
-rw-rw-r--  1 ubuntu ubuntu        2078 Aug  8 15:21 vss_demo.tar.gz
```

To ensure all files were downloaded successfully, run the following command and check the hashes match as shown in the image below.

```
Shell
sha256sum ~/vss_demo_downloads/*.tar.gz
```

```
190a73cb2857c1531d04a6a2dccf0700a2760e92d5932ede174fafe50327076c  alert-inspector-ui.tar.gz
71a84a0055f90b1aa94c6ae89da1cfd6faf9befca7d79095f1a73fde61e10ad4  Cosmos-Reason1.1-7B.tar.gz
0e20f1be7f2eef915efd4ee43c805517ea007cfea2a74af9b37bac2deab3aee6  cv-ui.tar.gz
56370a29363b4e54a014c8e75ce61d8bfa8c1505d07296a620eca26e1946c973  nv-cv-event-detector.tar.gz
7d9b10f6ed9e496518d6f3ce7a7435e53624d1dd5626f1a1e9fe81ada699927f  via-engine.tar.gz
bb7d1985411071118fae87c1704370663dcd6ddb087a82686851e2c1c830cd84  vss_demo.tar.gz
```

Load docker images

Using the docker load command, the four containers need to be added to your docker registry. The docker load commands will take about 10 minutes to complete.

```
Shell
cd ~/vss_demo_downloads
docker load -i alert-inspector-ui.tar.gz
docker load -i cv-ui.tar.gz
docker load -i nv-cv-event-detector.tar.gz
docker load -i via-engine.tar.gz
```

Verify all four docker images have been loaded.

```
Shell
docker images
```

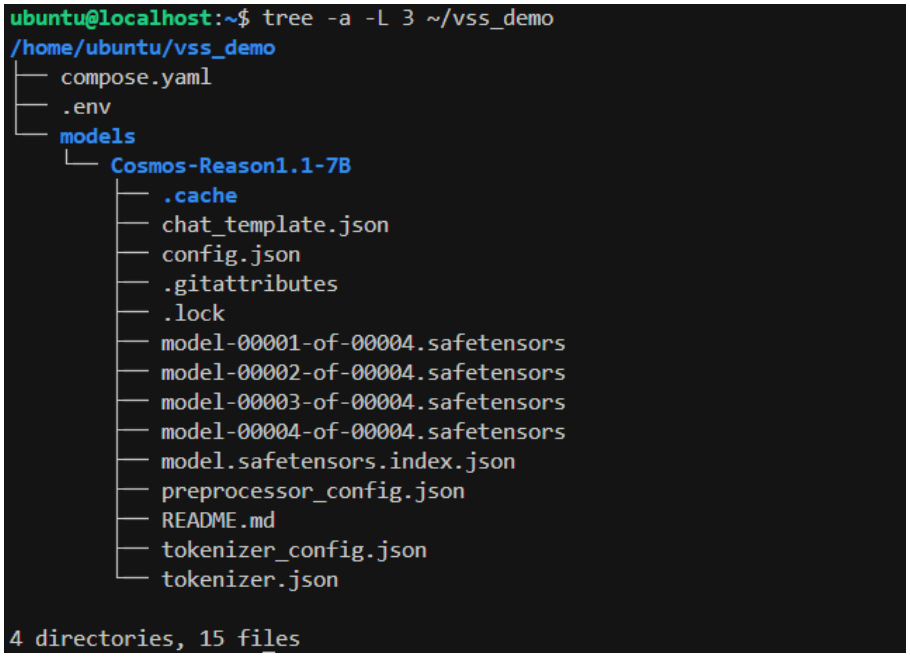
```
ubuntu@localhost:~/vss_demo_downloads$ docker images
REPOSITORY                                                        TAG                                  IMAGE ID       CREATED         SIZE
gitlab-master.nvidia.com:5005/via/cv-event-detector/nv-cv-event-detector  nv-cv-event-detector-main-5a07f40-public  a9d853970952   20 hours ago    21.7GB
gitlab-master.nvidia.com:5005/via/via-engine                      via-engine-main-9170c8c-public       5e6f5ee70c12   29 hours ago    33.3GB
gitlab-master.nvidia.com:5005/via/vlm_as_judge/cv-ui              main-e0746a9                         0e65c01263f9   37 hours ago    25.3GB
gitlab-master.nvidia.com:5005/via/via-engine/alert-inspector-ui   main-e4c3951                         17266b75186d   7 days ago      25GB
```

Setup demo folder

The model weights and configuration files need to be extracted into a vss_demo folder. Extracting the model weights will take 3-5 minutes to complete.

```
Shell
cd ~/vss_demo_downloads
tar -xzf vss_demo.tar.gz -C ~/
tar -xzf Cosmos-Reason1.1-7B.tar.gz -C ~/vss_demo/models/
```

Verify you have a ~/vss_demo directory with the following contents:



Inside the ~/vss_demo folder is a .env. Open this file and verify the MODEL_ROOT_DIR and MODEL_PATH environment variables are pointing to valid paths to access the Cosmos-Reason1.1-7B model weights. No change is needed if your user is set to “ubuntu”.

Run Demo

Docker compose will be used to run the demo. This will bring up all the containers to run the web UIs, computer vision pipeline and VSS.

Computer Vision UI	http://<jetson_ip>:7862/
VSS UI	http://<jetson_ip>:7860/

Web UI Ports

```
Shell
cd ~/vss_demo
docker compose up
```

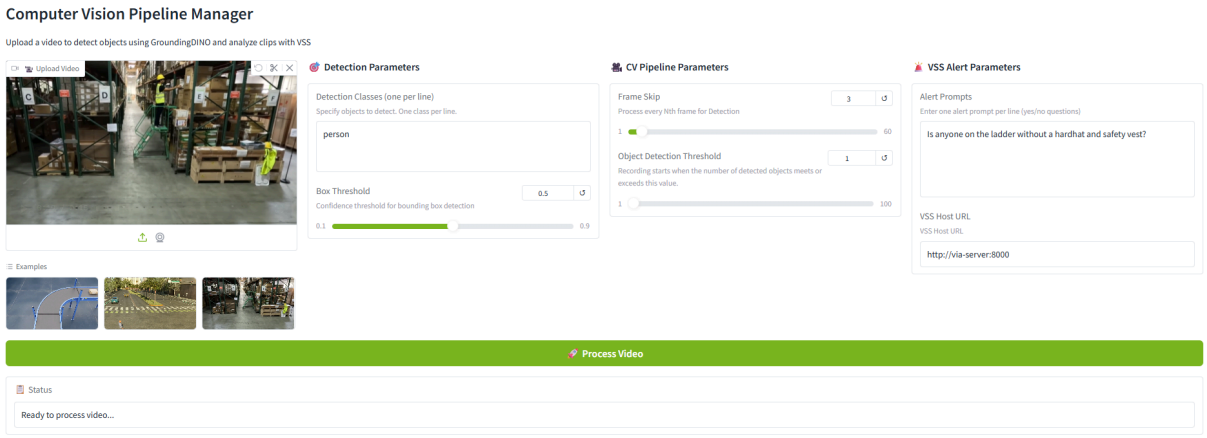
It may take 15-20 minutes to fully load the first time it is launched.

Once the CV UI has launched, the demo should be fully loaded. Check your terminal output for the following lines to verify the CV UI has launched.

```
None
cv-ui-1                  |
cv-ui-1                  | * Running on local URL:  http://0.0.0.0:7862
cv-ui-1                  | * To create a public link, set `share=True` in
`launch()`.
```

First go to the DeepStream CV Pipeline UI which will be running at port 7862. This can be accessed directly from a web browser on the Jetson or a separate system on the same local network.

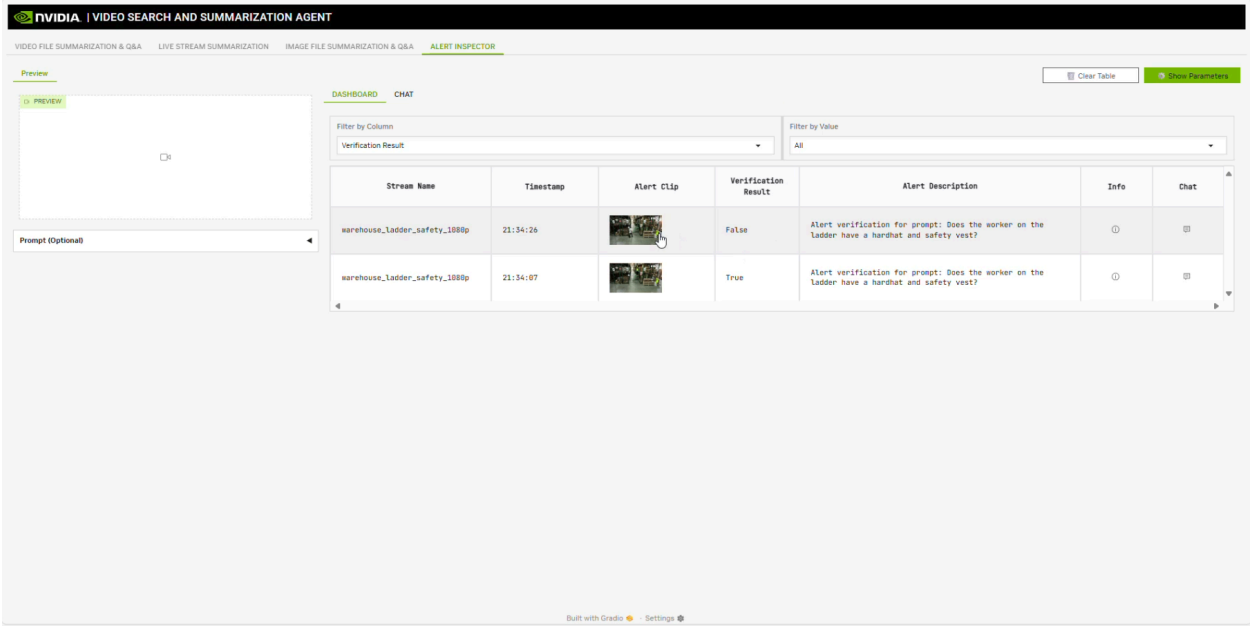
http://<jetson-ip>:7862



From this UI, you can select an example video or upload your own. This video will be processed as a stream by a GroundingDino based detection pipeline with DeepStream. The output of this pipeline will be short clips based on the detection class and threshold. If the number of objects detected in the video is greater than the object detection threshold, then the clip will be recorded and sent to VSS for further inspection. Here you can also set alert rules to be evaluated by VSS on the output clips. These alert rules are simple yes/no questions for VSS to evaluate.

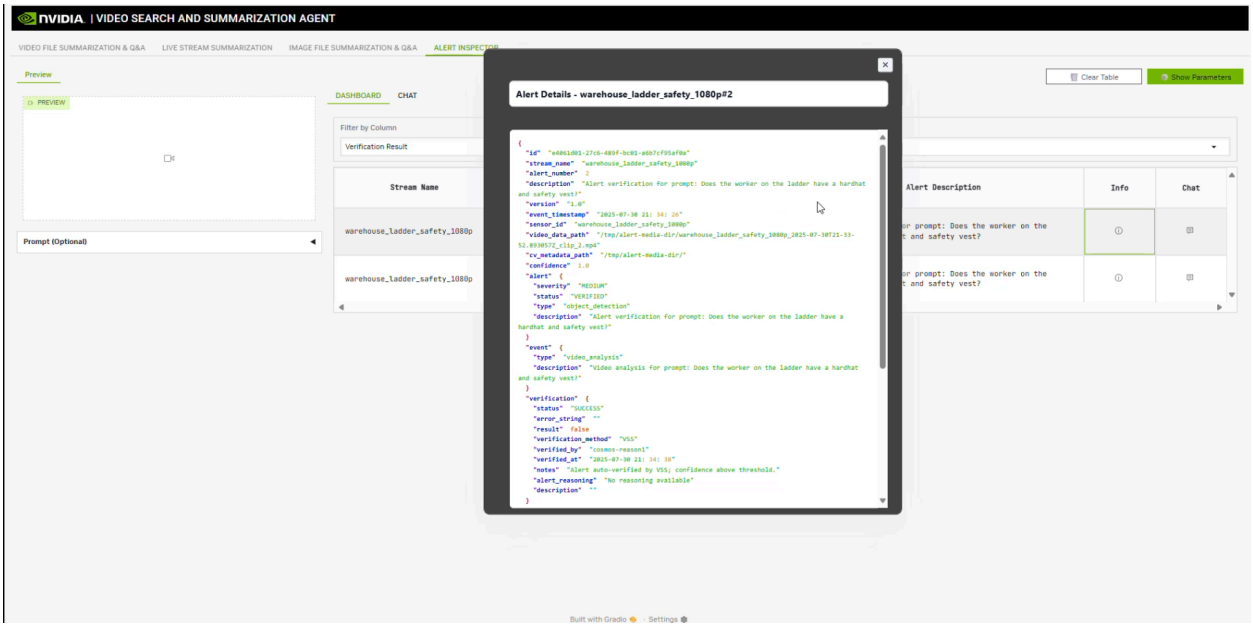
While the video is being processed by the detection pipeline, access the VSS UI at port 7860.

http://<jetson-ip>:7860

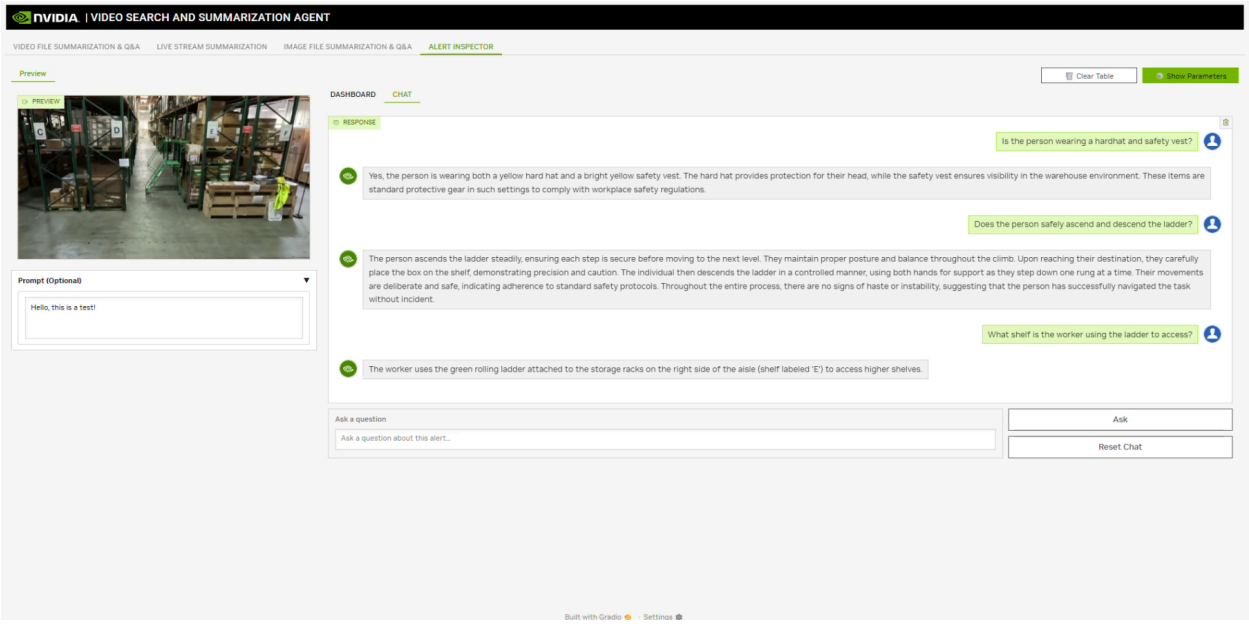


Here you will see the alerts pop up in real time as they are processed by the detection pipeline and sent to VSS for alert evaluation. In this case, we receive alerts to verify a person on a ladder is wearing their required hardhat and safety vest.

Each alert clip can be further inspected to view the full details such as the timestamps, alert prompts, VLM response and extra metadata.



You can also chat with each clip to ask follow up questions by clicking on the chat icon in each row.



The demo is preloaded with three examples. When any of the examples are selected in the CV UI, it will automatically fill in the associated parameters and alerts. For Q&A, here are some suggested questions:

Conveyor Belt Inspection

- Describe the condition of the box.
- Describe the layout of the conveyor belt.
- What is the color of the box?
- When did the box appear on the conveyor belt?

Warehouse Ladder Safety

- What is the worker wearing?
- Which aisle is the worker climbing to?
- Describe the warehouse setting.
- Is the worker wearing PPE?

Crosswalk Verification

- Is the person in danger?
- Which car goes close to the person?
- Is there a school bus in the scene?
- Is the person using the crosswalk?

Custom Video Guidelines

In addition to the three example videos provided, you can upload custom videos to the CV UI to test the pipeline on new objects and alerts.

Here are general guidelines for using custom input videos

- Use a 1080x1920, 30FPS, mp4 video file
- The video should have clear, detectable objects
- Tune the Box Threshold in the CV UI to control the sensitivity of the detections
- Alert prompts should be phrased as simple yes/no questions. Include any important details to help the model understand the question being asked.
- The frame skip parameter works best with values between 2-4. If set to 1, it may not leave enough GPU time for the VLM to process alerts with low latency.

RUN THE GENERATIVE AI BENCHMARKS

NVIDIA Jetson Thor delivers a giant leap in generative reasoning, powered by the latest NVIDIA Blackwell GPU architecture and 128 GB of memory. It offers an unmatched 2070 FP4 TFLOPS of AI compute within a 130 W power envelope, achieving up to 7.5x higher AI performance and 3.5x better energy efficiency than Jetson Orin. The Jetson Thor software stack supports all popular AI frameworks and most popular Generative AI models as covered in the Reviewers Guide.

Let us dive into the step-by-step guide for benchmarking LLM and VLM models using vLLM on the Jetson Thor Developer Kit.

Note that We recommend you reboot the developer kit to ensure we are starting from the clean slate and no other past processes are holding on to any memory.

1. Set to MAXN mode

```
Shell
sudo nvpmodel -m 0
```

2. Download the VLLM Container

    - Option 1: You can do “wget” to download the container from the NVIDIA server. If the download is failing, you can try the second option below.On your Jetson Thor, open a browser and download the vLLM container tar file from the location below.

```
Shell
wget
https://international.download.nvidia.com/JetsonThorReview/thor_vllm_container.
tar.gz
```

    - Option 2: On your Jetson Thor, open a browser and download the vLLM container tar file from the Google Drive location below: https://drive.google.com/file/d/1dZ_tGtyTmWx_Mycsm9BAbzGpZ9cKw_I0/view?usp=sharing

3. Load the VLLM Container
   Open a terminal and load the container from the tar file

```
Shell
sudo docker image load -i thor_vllm_container.tar.gz
```

Note that loading the docker image from the tar.gz file will take a few minutes.

4. Start the VLLM Serving Terminal

   Call the terminal opened in Step 2 as ("serving terminal"). Start the vllm container in which we will do model serving.

```
Shell
sudo docker run --ipc=host --net host --gpus all --runtime=nvidia --privileged
-it --rm -u 0:0 --name=vllm thor_vllm_container:25.08-py3-base
```

   In the container, switch to the scripts directory

```
Shell
cd scripts/
```

5. Start the VLLM Benchmark Terminal

   Open another terminal ("benchmark terminal") and launch the container again using the commands below.

```
Shell
sudo docker exec -it vllm /bin/bash
```

   Switch to the scripts directory, again.

```
Shell
cd scripts/
```

   Note: For every model benchmarking, please repeat steps 6, 7 and 8. Once you are done with benchmarking a model, kill the vllm serving started in Step 5 (by doing ctrl+c), before repeating steps 5, 6 and 7 for the next model.

6. Serving the Model (Serving Terminal)
   Execute below command in the vllm container running in "Sevring Terminal" from Step 4. And wait for an output similar to below image which indicates that the server has started.

```
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

For LLMs:

```
Shell
./run_vllm_llm_serve.sh <model_name>
```

For VLMs:

```
Shell
./run_vllm_vlm_serve.sh <model_name>
```

Example

```
Shell
./run_vllm_llm_serve.sh RedHatAI/Meta-Llama-3.1-8B-Instruct-quantized.w4a16
```

Note: A list of “model_name” is provided in the table below for LLMs and VLMs

7. Warm Up the Model (Benchmark Terminal)

During the warm up run, we will run a warm up benchmark session. This will help vllm build up various caches and benefit from techniques like prefix catching during benchmarking. We neglect the numbers from this warm up run.

Execute below in the vllm container running in “Benchmark Terminal” from Step 5.

For LLMs

```
Shell
./run_vllm_llm_warmup_run.sh <model_name>
```

For VLMs

```
Shell
./run_vllm_vlm_warmup_run.sh <model_name>
```

Example

```
Shell
./run_vllm_llm_warmup_run.sh RedHatAI/Meta-Llama-3.1-8B-Instruct-quantized.w4a16
```

8. Run the Benchmark (Benchmark Terminal)

Execute below in the vllm container running in "Benchmark Terminal" from Step 5.This is the same terminal where you ran the warm up run in Step 7.

For LLM

```
Shell
./run_vllm_llm_benchmark.sh <model_name> <max_concurrency>
```

For VLM

```
Shell
./run_vllm_vlm_benchmark.sh <model_name> <max_concurrency>
```

Example

```
Shell
./run_vllm_llm_benchmark.sh RedHatAI/Meta-Llama-3.1-8B-Instruct-quantized.w4a16
8
```

Sample Output

Maximum request concurrency: 8
100%|[redacted]| 50/50 [00:39<00:00, 1.26it/s]
============ Serving Benchmark Result ============
Successful requests: 50
Benchmark duration (s): 39.62
Total input tokens: 102350
Total generated tokens: 6227
Request throughput (req/s): 1.26
Output token throughput (tok/s): 157.17
Total Token throughput (tok/s): 2740.44
----------Time to First Token----------
Mean TTFT (ms): 231.97
Median TTFT (ms): 241.34
P99 TTFT (ms): 349.38
-----Time per Output Token (excl. 1st token)------

Mean TPOT (ms): 47.91
Median TPOT (ms): 47.86
P99 TPOT (ms): 58.96
----------Inter-token Latency----------
Mean ITL (ms): 47.50
Median ITL (ms): 46.83
P99 ITL (ms): 111.28
----------End-to-end Latency----------
Mean E2EL (ms): 6099.71
Median E2EL (ms): 6330.52
P99 E2EL (ms): 6441.85
==========

In the benchmarking table we published, we have used “Output token throughput (tok/sec)” which is Number of generated tokens per second (excludes prompt tokens) – key measure of generation speed.

Key Parameters

- Input Sequence Length: 2,048 tokens
- Output Sequence Length: 128 tokens
- Quantization: W4A16 (4-bit weights, 16-bit activations)
- Max Concurrency: Number of parallel requests (e.g., 8 for standard benchmarks)

Tip:
Run serving and benchmarking in two separate containers and terminals. Warm up models before benchmarking for optimal results (prefix caching).
You’re now set up to run VLLM benchmarks—just insert your model name from below table and choose the concurrency you want to test.

Model	**Model Name (To use with commands above)**
Large Language Models (LLM)	
Llama 3.2 3B	espressor/meta-llama.Llama-3.2-3B-Instruct_W4A16
LLama 3.1 8B	RedHatAI/Meta-Llama-3.1-8B-Instruct-quantized.w4a16
Llama 3.3 70B	RedHatAI/Llama-3.3-70B-Instruct-quantized.w4a16
Qwen 3 4B	RedHatAI/Qwen3-4B-quantized.w4a16

Qwen 3 8B	RedHatAI/Qwen3-8B-quantized.w4a16
Qwen 3 30B-A3B (MOE)	RedHatAI/Qwen3-30B-A3B-quantized.w4a16
Qwen 3 32B	RedHatAI/Qwen3-32B-quantized.w4a16
DeepSeek R1 Distill Qwen 7B	RedHatAI/DeepSeek-R1-Distill-Qwen-7B-quantized.w4a16
DeepSeek R1 Distill Qwen 32B	RedHatAI/DeepSeek-R1-Distill-Qwen-32B-quantized.w4a16
Vision Language Models (VLM)	
Qwen 2.5 VL 3B	RedHatAI/Qwen2.5-VL-3B-Instruct-quantized.w4a16
Qwen 2.5 VL 7B	RedHatAI/Qwen2.5-VL-7B-Instruct-quantized.w4a16
Qwen 2.5 VL 32B	Qwen/Qwen2.5-VL-32B-Instruct-AWQ
Llama 4 Scout 17B-16E (MoE)	RedHatAI/Llama-4-Scout-17B-16E-Instruct-quantized.w4a16

APPENDIX

1. **vLLM Instructions**
    - Download the container:
        - Option 1: You can do “wget” to download the container from the NVIDIA server. If the download is failing, you can try the second option below.On your Jetson Thor, open a browser and download the vLLM container tar file from the location below.

```
Shell
wget https://international.download.nvidia.com/JetsonThorReview/thor_vllm_container.tar.gz
```

        - Option 2: On your Jetson Thor, open a browser and download the vLLM container tar file from the Google Drive location below: https://drive.google.com/file/d/1dZ_tGtyTmWx_Mycsm9BAbzGpZ9cKw_l0/view?usp=sharing

    - Open a terminal and load the container from the tar file:

```
Shell
docker image load -i thor_vllm_container.tar.gz
```

    - Start the container:

```
Shell
sudo docker run --ipc=host --net host --gpus all --runtime=nvidia --privileged -it --rm -u 0:0 --name=$NAME thor_vllm_container:25.08-py3-base
```

2. **SGLang Instructions**
    - Option 1: You can do “wget” to download the container from the NVIDIA server. If the download is failing, you can try the second option below.On your Jetson Thor, open a browser and download the vLLM container tar file from the location below.

```
Shell
wget https://international.download.nvidia.com/JetsonThorReview/thor_sglang_container.tar.gz
```

- Option 2: On your Jetson Thor, open a browser and download the SGLang container tar file from the Google Drive location below: https://drive.google.com/file/d/1BI0e_589RTMlHVfgWDSXpxstLqqHFEv-/view?usp=drive_link

- Open a terminal and load the container from the tar file:

```
Shell
docker image load -i thor_sglang_container.tar.gz
```

- Start the container:

```
Shell
sudo docker run --ipc=host --net host --gpus all --runtime=nvidia --privileged
-it --rm -u 0:0 --name=$NAME thor_sglang_container:25.08-py3-base
```

3. Pytorch Instructions

- Option 1: You can do “wget” to download the container from the NVIDIA server. If the download is failing, you can try the second option below.On your Jetson Thor, open a browser and download the vLLM container tar file from the location below.

```
Shell
wget
https://international.download.nvidia.com/JetsonThorReview/thor_pytorch_contain
er.tar.gz
```

- Option 2: On your Jetson Thor, open a browser and download the Pytorch container tar file from the Google Drive location below: https://drive.google.com/file/d/1TqAtbHsPkNFoY-s2YKPsX1tJpH2l8K8w/view?usp=drive_link

- Open a terminal and load the container from the tar file:

```
Shell
docker image load -i thor_pytorch_container.tar.gz
```

- Start the container:

```
Shell
sudo docker run --ipc=host --net host --gpus all --runtime=nvidia --privileged -it --rm -u 0:0 --name=$NAME
thor_pytroch_container:25.08-py3-base
```

4. **LLama.cpp Instructions**
   On your Jetson Thor, compile Llama.cpp by following the command below.

   - Clone the repository

```
Shell
git clone https://github.com/ggml-org/llama.cpp
cd llama.cpp
```

   - Build Llama.cpp

```
Shell
sudo apt-get install libcurl4-openssl-dev


cmake -B build -DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES="110"

# Set this environment variable
GGML_CUDA_ENABLE_UNIFIED_MEMORY=1

cmake --build build --config Release -j
```

5. **Camera Integration**

   If you would like to run your own workflow that involves a camera, JetPack 7.0 supports RealSense true depth D455 and D435i robotics cameras.

   Note that as of writing Realsense may not have yet released debian packages that officially supports Jetpack 7. However, it is possible to build v2.56.3 of librealsense from source on the Jetson and be usable.

   1. Install Dependencies
      sudo apt update && sudo apt install -y git cmake build-essential libssl-dev libusb-1.0-0-dev pkg-config libgtk-3-dev libglfw3-dev libgl1-mesa-dev libglu1-mesa-dev wget python3 python3-dev

   2. Download and Modify Build Script

wget
https://raw.githubusercontent.com/jetsonhacks/installRealSenseSDK/refs/heads/master/buildLibrealsense.sh && chmod +x buildLibrealsense.sh
sed -i 's/-DFORCE_LIBUVC=ON/-DFORCE_RSUSB_BACKEND=true/g' buildLibrealsense.sh

3. Build and install version v2.56.3 RealSense SDK
   ./buildLibrealsense.sh -v v2.56.3 -n
   If successful, after a few minutes, you should expect a message in green saying Library Installed

4. Reload and run udev rules
   sudo udevadm control --reload-rules && sudo udevadm trigger

5. Test Installation and detection of Realsense cameras. Connect your RealSense camera and run:
   rs-enumerate

6. Test camera you can view all streams. This is a GUI application so you need a display.
   realsense-viewer

   Note: If you have issues, please contact Realsense at https://github.com/IntelRealSense/librealsense/issues

RESOURCES

1. Jetson T5000 Series Modules Data Sheet
2. Other Jetson Thor Technical Documents
3. NVIDIA Isaac GR00T
4. NVIDIA Video Search and Summarization Agent
5. NVIDIA Jetson AI Lab
6. NVIDIA Developer Forum

Notice

ALL INFORMATION PROVIDED IN THIS REVIEWER'S GUIDE, INCLUDING COMMENTARY, OPINION, NVIDIA DESIGN SPECIFICATIONS, REFERENCE BOARDS, FILES, DRAWINGS, DIAGNOSTICS, LISTS, AND OTHER DOCUMENTS (TOGETHER AND SEPARATELY, "MATERIALS") ARE BEING PROVIDED "AS IS." NVIDIA MAKES NO WARRANTIES, EXPRESSED, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO MATERIALS, AND EXPRESSLY DISCLAIMS ALL IMPLIED WARRANTIES OF NONINFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A PARTICULAR PURPOSE.

Information furnished is believed to be accurate and reliable. However, NVIDIA Corporation assumes no responsibility for the consequences of use of such information or for any infringement of patents or other rights of third parties that may result from its use. No license is granted by implication or otherwise under any patent or patent rights of NVIDIA Corporation. Specifications mentioned in this publication are subject to change without notice. This publication supersedes and replaces all information previously supplied. NVIDIA Corporation products are not authorized for use as critical components in life support devices or systems without express written approval of NVIDIA Corporation.

Trademarks

NVIDIA, the NVIDIA logo, CUDA, Jetson Orin, Jetson Thor, NVIDIA Cosmos, NVIDIA DGX, NVIDIA Isaac Sim, NVIDIA JetPack, NVIDIA OGX, NVIDIA Omniverse, NVIDIA Xaver, and TensorRT are trademarks and/or registered trademarks of NVIDIA Corporation in the U.S. and other countries. All rights reserved. Other company and product names may be trademarks of the respective companies with which they are associated.

Copyright